Error Propagation in Multi-Step Agentic AI Workflows for Research Automation

Sutariya Gaurav Karsanbhai
MCL2026003

Abstract

Using multi-step AI workflows can help streamline literature reviews by allowing researchers to break down their research. Different agents can be used to create search queries, search for papers using a service like Crossref or arXiv, filter sources, extract evidence, run code, compare results, and create a manuscript. However, there are concerns regarding the reliability of the final report. If an error occurs at an early stage, subsequent agents will believe the assumption to be true and use it in their analysis. This review will discuss the error propagation in research focused agent systems. Six different areas will be reviewed (ReAct, Tool use (Toolformer and ToolLLM), RAG (Retrieval Augmented Generation) & Self-RAG, Multi-agent (AutoGen and MetaGPT), Reflexion, and Trajectory based (AgentBench, WebArena, and GAIA). Errors can be categorised into 8 categories: Planning, Information retrieval, Filtering, Evidence extraction, Reasoning, Citing, Tool use, and Memory. This review will discuss how errors propagate and how they can be mitigated. There are several issues that have not been overcome including: There being no universal standard way to measure error propagation, identifying the root causes, verification, reproducible online experiments, and error injection benchmarks. It can be difficult to determine where an error has occurred in the final document due to a lack of evaluation with the chain of evidence.

Keywords: Agentic AI, Large Language Models, Research Automation, Error Propagation, AI Agents, Reliability, Tool Use, Citing, Multi-step, Human-in-the-loop

1. Introduction

Large language models are typically used to answer prompts, however many use a multi-step approach by placing the model at the centre of a software workflow. This allows the model to break down the task at hand into smaller steps, search external indexes, run code, and perform multiple actions. Many current systems use a multi-step approach by placing the model in the centre of a software workflow. The model will break the general goal into smaller tasks. They will search external indexes, run code, and store notes. ReAct was introduced by Yao et al. in 2023 [1] and helped popularise this by using reasoning and actions. Other approaches have been created focusing on creating more explicit intermediate steps such as Chain-of-Thought prompts [2], Self consistency [3], and Tree of Thoughts [4]. Whilst this can allow the agent to be transparent with it's intermediate steps it does not guarantee that the logic is correct. Later steps will need to validate the important points and calculations.

Automated literature reviews can be a complex area. A research pipeline may generate queries, search for papers within academic indexes, determine relevance, extract evidence, compare methods, perform statistical tests, and generate a citing document. A research workflow may generate queries, search for papers within academic indexes, determine relevance, extract results, compare methods, perform statistical tests, and generate a citing report. Using this can help save time on performing repetitive tasks within the library. However the research conclusions are based on a chain of evidence. If an error occurs in any of the steps, it can be passed onto the subsequent steps and believed to be reliable. This could cause issues with the wider research if the subsequent researchers rely on this review.

This paper focuses on error propagation. A common issue is when the output from one module is assumed by the next. An example of this is when an error occurs during the search step. The model will retrieve an irrelevant paper due to overlapping terminology with the topic. The summariser will provide an accurate summary, the synthesis will compare this summary to relevant studies and the writing will add a valid citation. The final document will appear to be valid despite the lack of evidence to back up the argument. Other issues can occur when the agent misreads the table, fails to consider an important limitation, or cites a real paper to back up a claim that has not been tested. Whilst verifying that there are references, there is no guarantee that they support the statements.

This review will look at many aspects of error propagation including agent reasoning, API use, RAG, multi-agent collaboration, self correction, and multi-step evaluation. Four questions will be answered:

1. Where do errors occur initially in an agent driven research workflow?
2. How do local errors propagate between modules and become more impactful?
3. What engineering solutions can be implemented to detect, prevent, or mitigate these errors? What are the limitations?
4. What research issues remain to be resolved to trust automated research?

This paper will start by looking at other systems and evaluation methods. It will then categorise the sources of errors, explain how they propagate, and give an example of how an error would propagate through a 7 step literature review. Later sections will discuss how errors can be mitigated, current limitations, and areas of focus for future research.

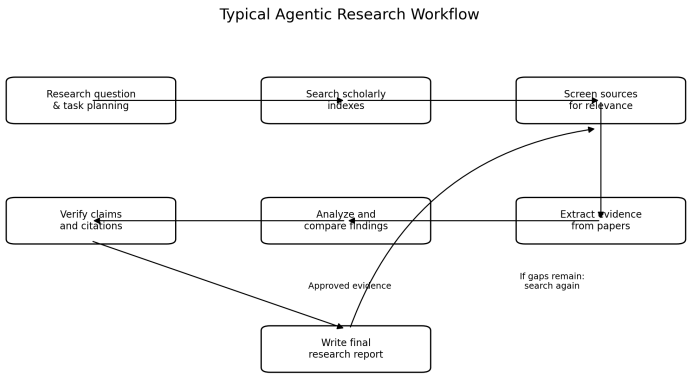


Figure 1: Typical steps involved in an agentic research workflow with feedback for further searching.

2. Background and Related Work

2.1 Agentic AI & Multi-step Reasoning

Single-turn systems will provide an answer to the prompt given. However agentic systems will perform a planning, act, observe, and update step multiple times. ReAct [1] helps by combining reasoning and external tools. The agent will update their plan depending on the observations. In a research scenario this could include searching an index, reading abstracts, validating results using code, and searching for cited work. The extra steps mean more handoffs can occur leading to loss of context or assumptions not supported by evidence. Flexible actions can improve capabilities but make it harder to ensure reliability.

2.2 Tool Enhanced Language Models

LLMs can often struggle with mathematical calculations, structured data, and live database access. Toolformer [5] looked into how a model can learn when to use tools such as calculators, search engines, and calendar APIs. ToolLLM [6] has expanded this to include a large range of real world REST APIs. This means selecting the right endpoint and creating the correct arguments can be learned. While using tools can help reduce some guessing, the agent may still make errors by choosing the wrong endpoint, providing invalid queries, and incorrect units. How the tools are used, parsed, and the information provided will impact the reliability of the full workflow.

2.3 Retrieval-Augmented / Grounded Generation

Retrieval-Augmented Generation (RAG) provides a way to link generation to documents found within an external collection [7]. This can help to provide more evidence-based answers as the model has the source text available. Self-RAG [8] builds on this by adding a learned method to determine when to search and critique the content. This can be important in research automation as the agent may find a real paper/DOI but it is important to ensure that the cited section supports the claim. Grounding can help reduce reliance on unsupported recollection, however, it will not prevent poor searching, misreading, or incorrect interpretation.

2.4 Reflection

Reflexion [9] allows the agent to improve by using a form of episodic memory. Previous attempts will provide text based feedback. Within research this could include noticing that a paper is irrelevant, or two extracted values don't match. This will help to update the notes to avoid error propagation. The effectiveness of this depends on how well the critique is. If it is receiving incomplete feedback or not identifying the correct issue, it could result in the original error being kept or the conclusion becoming even more confident. This should be considered to be a form of self correction and not independent verification.

2.5 Multi-agent Frameworks

It can become complex to manage a single prompt. Multi-agent frameworks will split the tasks between agents. AutoGen [10] allows agents, humans, and tools to interact. MetaGPT [11] has a more structured approach using roles and processes inspired by software engineering. Other work includes Generative Agents [12] which looks at long-term memory and social habits of agents, Voyager [13] which creates a library of reusable skills whilst exploring Minecraft, and ReSpAct [14] which combines reasoning, conversation, and action for interactive assistants. Whilst splitting the tasks between agents can help to organise this does not guarantee independent verification. If the agents are using the same model, context, or faulty summary they may share the same blind spots.

2.6 LLM Agent Benchmarking

Traditional language model benchmarks often focus on individual answers. However agent benchmarks are starting to focus more on sequences of actions in interactive environments. AgentBench [15] studies how models perform in 8 scenarios, highlighting issues with long-term planning, sequential reasoning, and instruction following. WebArena [16] tests browser agents on realistic websites, Mind2Web [17] tests general purpose web agents on real world sites, and GAIA [18] includes multi-step tasks for assistants requiring practical reasoning and tool use. SWE-bench [19] measures whether the model can resolve real GitHub issues. WebGPT [20] looked into browser aided answers with support from page evidence and human feedback. Overall these show how individual

scores are not enough and how an agent may perform well in a single action but fail due to errors and dependencies.

3. Common Error Sources in Research Based Agentic Pipelines

A research automation pipeline can be thought of as having a series of connected steps. As each step often takes the output from the last step it can be easy for an error to occur. There are three main categories of errors: Input errors which will cause distortion in the task, Process errors which will affect the evidence and reasoning, and Output errors which will relate to citing and long-term memory.

Table 1: Common error sources in research focused agentic workflows.

Error Source / Step	Example Error	Impact
Planning	Misunderstanding of research question	Important sub questions are not investigated.
Information Retrieval	Misses key studies or gets irrelevant papers	Evidence is incomplete/irrelevant.
Filtering	Poor/mismatched sources	Poor quality evidence.
Evidence Extraction	Wrong row/metric/limitation	Misrepresents findings.
Reasoning	Incompatible results are compared	Overstates the evidence.
Citing	Links real paper to unsupported claim	Unable to verify argument.
Tool Use	Wrong arguments/units provided	Incorrect calculations/data.
Memory	Saves faulty summary	Same error occurs in future tasks.

3.1 Planning Errors

The planning determines what will be investigated in the workflow. If the planner misunderstands the prompt, or breaks down the task into imbalanced sub questions, the subsequent steps may complete their task correctly but the question will not be answered. When assessing AI research assistants it is important to consider how well they search, reason, cite, tool use, and human oversight. If the plan only focuses on the benchmark scores then other areas may be ignored.

3.2 Information Retrieval Errors

Queries may be too narrow and miss key studies. Or too broad and provide irrelevant/old papers. Duplicated records can cause a lot of noise, the indexes may be out of date, and simple parsers may extract short snippets. This can cause issues if the filtering modules assume all results are relevant.

3.3 Filtering Errors

Broad searches may not provide reliable papers. The filtering modules will need to consider if the source really answers the research question, and if the methods are strong enough for comparison. Prominent titles can be distracting with a small sample, poor baseline, and incorrect population. Poor sources will pass through and irrelevant results will be included in the evidence.

3.4 Evidence Extraction Errors

Even if the paper is relevant and peer reviewed it can be misrepresented. The agent may extract values from the baseline row of a busy PDF, misunderstand if the hypothesis or result, miss out on a caveat, and misread table columns. They may also over generalise from a small example. The summary may still be well written with a real citation, meaning there is little reason for the subsequent modules to doubt it unless they compare it to the passage.

3.5 Reasoning Errors

Even if the evidence has been extracted accurately the analysis may not be sound. The reasoning module may wrongly assume correlation/causation, ignore base rates, and compare incompatible metrics. Whilst using techniques like Chain-of-Thought prompts [2] can make the reasoning more transparent, it does not guarantee that the conclusion can be drawn from the evidence. Important inferences will need to be checked against data, assumptions, and methods used in cited studies.

3.6 Citing Errors

Errors may occur with the author names, year published, venue, page numbers, or DOI. A more subtle error can occur if the cited work exists but doesn't back up the sentence. To ensure reliable citing the reference must be checked to ensure it is real, and accurately described. It also needs to be checked to ensure the source does provide evidence.

3.7 Tool Use Errors

Calculators, Python environments, PDF parsers, and REST APIs can help agents complete tasks outside of the language model [5,6]. However this can cause errors. The module may use the wrong function, provide invalid arguments, and misinterpret the returned schema. Only when inputs, outputs, and status are validated will the tool use be reliable.

3.8 Memory Errors

Many long running systems will keep notes, summaries, claims, and decisions in a scratchpad or vector store. If an incorrect statement is added to this it may be retrieved and used multiple times in future tasks. This means the error will be sustained and may affect future work. This becomes more of an issue as systems develop long term memories like those in Generative Agents [12], and reusable skills like those in Voyager [13].

4. Error Propagation

4.1 Sequential Propagation

The most straightforward error propagation is sequential propagation. An error will occur in the early stages causing an incorrect intermediate result. This will be used by later modules causing the final report to state an unsupported claim. An example is an irrelevant paper resulting in a misleading summary. This will be used in a flawed comparison and result in a weak conclusion.

Figure 2: A small error can occur in the early stages and lead to an unsupported conclusion if the intermediate outputs are not checked.

If there are n steps in the workflow with the probability of success for step i being p_i then the probability of success overall can be modelled as:

$$\mathbf{P(success) = \prod_{i=1}^{n} p_i}$$

Assuming all steps have a 0.95 probability of succeeding then a 10 step workflow would have a 0.95^{10} probability of succeeding. However real systems are more complex and the errors may not be independent. This does show how even if the components are performing well the overall reliability may be lower.

4.2 Amplification

An error may be amplified throughout the workflow. For example, if the filtering module includes a weak preprint which doesn't fully match the topic. The error will initially affect the reading list. However the summariser will reduce this to a concise statement. The synthesis module will include this alongside results from stronger papers. Finally the report writer will highlight this point. This has caused a small error in the filtering to become a core claim.

4.3 Masking

LLMs can generate high quality academic text even if the information they are given is incorrect. The incorrect note may be surrounded by clear sentences, formal headings, and sensible citations. This will make the output appear more reliable than the evidence. Fluency should not be confused with factual support.

4.4 Correlated Errors

Splitting tasks between multiple agents does not guarantee independent opinions. If the agents used in AutoGen [10] or MetaGPT [11] are using the same underlying model, context buffer, or have one faulty summary then they may all make the same error. This may appear to confirm the error when in reality it is coming from the same source. Independent verification will require meaningful differences in the evidence retrieval and checking.

4.5 Feedback Loop Errors

Whilst feedback loops can help correct errors, they can also cause them to be amplified. Reflexion [9] shows how text based feedback can help the agent improve. However if the critic does not identify the

issue (e.g. misreading the result is identified as a wording issue) then the next draft may include a more forceful incorrect conclusion. Only when there is evidence based critique will a feedback loop be useful.

5. Example of Error Propagation in Automated Research

Imagine we have a literature review which has a technical term in the original question that is being misinterpreted. This error will propagate through 7 steps:

1. Question interpretation – The term is misinterpreted by the planning module.
2. Search – Queries are created with the incorrect interpretation.
3. Filtering – Papers are selected which fit the incorrect definition.
4. Evidence extraction – Evidence is accurately extracted from the incorrectly selected papers.
5. Synthesis – Findings are synthesised to create a coherent comparison.
6. Citing – Real citations are added.
7. Report generation – A well written survey is created by the writing module.

Once the PDF has been retrieved, the references may be correctly placed and values extracted as per the tables in the paper. However, the report itself does not answer the correct question. Whilst checking the DOIs is a good start, a reliable audit of the research will need to follow the full path from question to source to evidence to claim to conclusion. This means the audit is susceptible to errors if any part of the chain is incorrect.

6. Mitigation Strategies

6.1 External Grounding

Using external indices and sources can give a factual grounding to the research. Using scholarly databases, publishing records, DOI databases, university repositories, datasets and technical documentation can all be useful sources. ReAct [1] and RAG [7] encourage the system to make use of external information to increase the reliability. However, this does not guarantee the system will retrieve the correct information and interpret it appropriately.

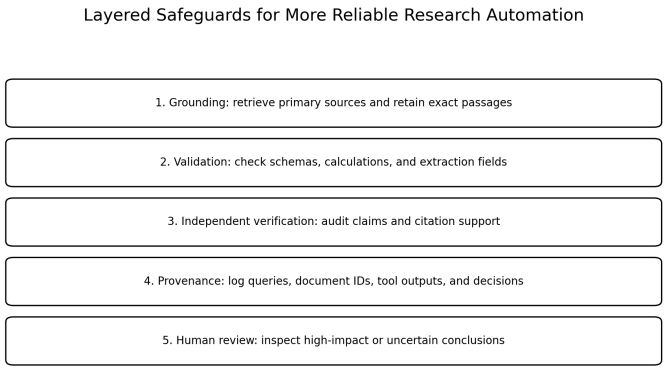


Figure 3 - Additional safeguards throughout the workflow.

6.2 Tool Use

Tools can be used for searching, mathematical operations, parsing PDFs and conducting statistical analysis. Overall, this follows the concept of Toolformer [5], ToolLLM [6]. Using tools can make certain aspects more reliable by using specific tools. However, the output should be audited. For example, ensure any p-values or effect sizes are traced back to the original input, calculation and the claim made in the manuscript.

6.3 Reflection

Reflection can be used to improve the draft. Using Reflexion [9], aspects such as whether the paper answers the question, whether quotes support the summary, and if any claims are unsupported can be asked. This can be made more reliable by ensuring the system refers back to the original sources. If not, the critic will have the same blindspots as the author.

6.4 Independent Verification

As much as possible, the authoring and verification steps should be kept separate. An independent verification can ensure the papers are cited, verify references, recalculate results, view the source material and ensure the claims are supported by the citations. Keeping the verification process separate from the authoring process can increase the probability that they don't make the same assumptions. This is especially useful when verifying high impact claims.

6.5 Provenance

Keeping track of the provenance ensures that we can see how a statement came to be in the report. For every important claim, the system should store the question, search query, document ID, relevant section of the document, extracted values, analysis and final statement. This can help identify the point of error.

6.6 Human-in-the-Loop

Having a human-in-the-loop is most useful when we need to make a decision. This can include having the researcher approve the plan, checking the shortlisted papers, resolving ambiguous interpretations, checking any major conclusions and handling uncertainties. This should not mean every token is checked by a human, but rather ensure the steps requiring expert opinions are.

7. Comparing Mitigation Strategies

Overall, each mitigation strategy helps us in different ways. External grounding improves evidence collection, tools can improve reliability of certain steps, reflection can help improve the draft, independent verification ensures claims are reliable, provenance helps diagnose errors and human-in-the-loop ensures domain knowledge is used in key decisions.

Overall, there are a few trends. Firstly, many of the strategies focus on retrieval and local reasoning, with little work being undertaken on citation verification and long-term memory hygiene. Secondly, using agent systems can easily organise the tasks but the errors can be correlated if the agents are sharing models or contexts. Finally, trajectory-based systems tend to focus on completion of the task rather than noticing and correcting errors. These will help inform the research questions.

Strategy	Core Mechanism	Key Focus	Typical Evaluation	Limitations
ReAct [1]	Alternates between reasoning and taking actions in an external environment.	Reasoning & Tools	Question Answering, Fact Checking, Interactive Tasks	Still prone to errors when interpreting the retrieved information.
Reflexion [9]	Stores textual reflections from failed attempts.	Reasoning & Self-correction	Sequential Decision Making, Coding, Logic Tasks	May not find the correct reason for failure.
RAG [7] & Self-RAG [8]	Conditions generation on retrieved text and Self-RAG introduces adaptive retrieval & reflection.	Retrieval & Citation Verification	Knowledge Intensive NLP Tasks	Dependent on retrieval and quality of passages.
Toolformer [5] & ToolLLM [6]	Learns to use external APIs and tools.	Tools	Tool Augmented Task Suites	Will have valid calls but may not interpret the results correctly.
AutoGen [10] & MetaGPT [11]	Uses agent roles to coordinate with other agents.	Planning & Division of labour	Multi-Agent & Software Engineering Tasks	Errors can be correlated due to shared models/contexts.
AgentBench [15] & WebArena [16] & GAIA [18]	Uses actions to evaluate performance in interactive environments.	Agent Evaluation	Multi-Environment Task Completion	May not find any issues during the process as only final score is given.

8. Research Challenges

8.1 No Standardised Error Propagation Metrics

Most leaderboard approaches only look at whether the agent completes the task. There is no standardised way to determine how the errors are propagated throughout the process. We cannot determine when the error first occurred, if it was noticed by subsequent modules or how much it altered the conclusion. More detailed metrics are required to ensure the pipeline is reliable. whilst citation and atomic fact evaluation benchmarks are useful, they do not tell us how the errors propagate throughout the research [21, 22].

8.2 Causation

If the review makes a false conclusion, the error may have occurred from the plan, search query, screening, PDF extraction, reasoning, memory or conclusion. Without the intermediate inputs and outputs, it is hard to know exactly what caused the issue. Having a useful audit trail will help trace back from the conclusion to the exact evidence and decisions made.

8.3 Long Horizon Reliability

Research can involve many steps. This could be up to dozens or hundreds of steps. Therefore, the system needs to make sure it keeps its state, ensures it is progressing and can recover from any errors. As shown in AgentBench [15], there are still issues with long horizon planning, sequential decision making and instruction following. This is exacerbated by each step depending on the previous.

8.4 Verification Issues

Using a language model for verification could cause issues. The model may share the same training data, prompt bias and weaknesses in reasoning as the model being verified. Therefore, it may approve the very error it was meant to catch. Verification should be done using primary sources, alternative retrieval and explicit testing.

8.5 Reproducibility

Rankings can change, websites can be removed, APIs change and preprint papers can be updated. To ensure the research can be reproduced at a later date, the system should save the search queries, when they were retrieved, document IDs, source text, model version, tool responses and decisions. This will make it easier to see why different results occurred.

9. Research Gaps

9.1 Whole Workflow Benchmarks

Future benchmarks should ensure whole research workflows are tested. Based on trajectory-based approaches like AgentBench, WebArena and GAIA [15, 16, 18], we can introduce specific errors at certain steps and see if the downstream modules can detect, isolate and correct them.

9.2 Causal Error Lineage

Having an explicit lineage graph showing the causal relationship between the original error and any intermediate summaries, claims and conclusions can help us trace where the error occurred and what impact it had on the final report. This could be used to develop a debugging tool to easily see what issues are occurring in complex agent workflows.

9.3 Independent Multi-Agent Verification

Simply having different agent roles will not ensure independent verification. The verification should have different prompts, search queries, direct access to primary sources, independent verification and explicit rules for when there is a disagreement. This will ensure they don’t share the same blindspots.

9.4 Evidence-Based

Many systems will write a sentence and then search for a citation. Evidence-based workflows will ensure the source is retrieved first, the passage is verified, relevant evidence extracted, claim formed and then cited. By requiring evidence to be recorded before the factual sentence is added to the draft, it can reduce unsupported statements. This will increase the grounding principles from Self-RAG [8] to full research reports.

9.5 Uncertainty Aware

Research findings aren’t always concrete. However, the generated text may give the impression that uncertain and well-supported findings are certain. The agents should include explicit support for

claims such as verified, strongly supported, partially supported, uncertain or unsupported. The uncertainty should be based on evidence with clear criteria.

9.6 Human-AI Collaboration

Fully autonomous research can be risky. Therefore, agents can be used for repetitive tasks such as researching, organising and formatting whilst the human researchers can make decisions about the question, evaluating methods, resolving ambiguous interpretations and approving major conclusions. This should ensure the research is controlled.

10. Conclusion

Multi-step agents can speed up research by searching for literature, filtering sources, extracting evidence, comparing between studies and producing reports. There are many useful building blocks available. ReAct uses reasoning and external actions [1], RAG and Self-RAG use retrieved text [7, 8], Reflexion uses textual reflection [9], AutoGen and MetaGPT organise tasks [10, 11] and benchmarks like AgentBench, WebArena and GAIA give evaluation across long tasks [15, 16, 18].

However, combining these can create reliability issues. As shown in the taxonomy and propagation mechanisms, an error can occur at any point during the planning, retrieval, filtering, extraction and reasoning stages. This results in the error becoming an accepted assumption across all subsequent steps. The error can then be exaggerated and improved upon by subsequent modules meaning the final paper appears to be well-supported when in reality it isn't. There is no single solution to this issue. External grounding won't stop misinterpretation, tools won't ensure correct use of results and reflection may share the same blindspots.

To ensure trust-worthy research automation, we need to ensure each step is verified, evidence is collected from primary sources, independent verification, detailed provenance, explicit uncertainty and human-in-the-loop at key decision points. We also need better metrics and lineage tools to ensure we can see where the errors occurred, how they propagated and if the workflow recovered. The goal is to ensure each important claim can be traced back to evidence and any failures can be identified.

These are proposed for future evaluation.

Proposed Metrics for Error Propagation

Metric	Definition
Stage Error Rate	Number/Proportion of new errors at each stage.
Propagation Rate	Proportion of errors from one stage that have an effect on subsequent modules.
Amplification Factor	Change in severity of the error between the original and the final report.
Recovery Rate	Proportion of errors introduced that have been identified and corrected.
Un-detected Error Rate	Errors that weren't detected.
Claim Support Accuracy	Proportion of claims that are genuinely supported by the cited sources.

References

[1] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). ReAct: Synergizing reasoning and acting in language models. International Conference on Learning Representations (ICLR). DOI: Open DOI / publication page

[2] Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q.V., & Zhou, D. (2022). Chain-of-thought prompting elicits reasoning in large language models. Advances in Neural Information Processing Systems, 35. DOI: Open DOI / publication page

[3] Wang, X., Wei, J., Schuurmans, D., Le, Q.V., Chi, E.H., Narang, S., Chowdhery, A., & Zhou, D. (2023). Self-consistency improves chain of thought reasoning in language models. ICLR. Link: Open DOI / publication page

[4] Yao, S., Yu, D., Zhao, J., Shafran, I., Griffiths, T., Cao, Y., & Narasimhan, K. (2023). Tree of thoughts: Deliberate problem solving with large language models. Advances in Neural Information Processing Systems, 36. DOI: Open DOI / publication page

[5] Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Hambro, E., Zettlemoyer, L., Cancedda, N., & Scialom, T. (2023). Toolformer: Language models can teach themselves to use tools. Advances in Neural Information Processing Systems, 36. DOI: Open DOI / publication page

[6] Qin, Y., Liang, S., Ye, Y., Zhu, K., Yan, L., Lu, Y., Cong, X., Tang, X., Qian, B., Zhao, S., Xie, R., Zhou, J., Gerstein, M., Liu, Z., & Sun, M. (2024). ToolLLM: Facilitating large language models to master 16,000+ real-world APIs. ICLR. Link: Open DOI / publication page

[7] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W-t., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. Advances in Neural Information Processing Systems, 33, pp. 9459-9474. DOI: Open DOI / publication page

[8] Asai, A., Wu, Z., Wang, Y., Sil, A., & Hajishirzi, H. (2024). Self-RAG: Learning to retrieve, generate, and critique through self-reflection. ICLR. DOI: Open DOI / publication page

[9] Shinn, N., Cassano, F., Berman, E., Gopinath, A., Narasimhan, K., & Yao, S. (2023). Reflexion: Language agents with verbal reinforcement learning. Advances in Neural Information Processing Systems, 36. DOI: Open DOI / publication page

[10] Wu, Q., Bansal, G., Zhang, J., Wu, Y., Li, B., Zhu, E., Jiang, L., Zhang, X., Zhang, S., Liu, J., Awadallah, A.H., White, R.W., Burger, D., & Wang, C. (2024). AutoGen: Enabling next-gen LLM applications via multi-agent conversation. Conference on Language Modeling (COLM). DOI: Open DOI / publication page

[11] Hong, S., Zhuge, M., Chen, J., Zheng, X., Cheng, Y., Wang, J., Zhang, C., Wang, Z., Yau, S.K.S., Lin, Z., Zhou, L., Wu, C., & Schmidhuber, J. (2024). MetaGPT: Meta programming for a multi-agent collaborative framework. ICLR. Link: Open DOI / publication page

[12] Park, J.S., O'Brien, J., Cai, C.J., Morris, M.R., Liang, P., & Bernstein, M.S. (2023). Generative agents: Interactive simulacra of human behaviour. Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology. DOI: Open DOI / publication page

[13] Wang, G., Xie, Y., Jiang, Y., Mandlekar, A., Xiao, C., Zhu, Y., Fan, L., & Anandkumar, A. (2023). Voyager: An open-ended embodied agent with large language models. arXiv. DOI: Open DOI / publication page

[14] Dongre, V., Yang, X., Acikgoz, E.C., Dey, S., Tur, G., & Hakkani-Tür, D. (2024). ReSpAct: Harmonising reasoning, speaking and acting towards building large language model-based conversational AI agents. arXiv. DOI: Open DOI / publication page

[15] Liu, X., Yu, H., Zhang, H., Xu, Y., Lei, X., Lai, H., Gu, Y., Ding, H., Men, K., Zhang, S., Deng, X., Zeng, A., Du, Z., Zhang, C., Shen, S., Zhang, T., Huang, M., Dong, Y., & Tang, J. (2024). AgentBench: Evaluating LLMs as agents. ICLR. DOI: Open DOI / publication page

[16] Zhou, S., Xu, F.F., Zhu, H., Zhou, X., Lo, R., Sridhar, A., Cheng, X., Ou, T., Bisk, Y., Fried, D., Alon, U., & Neubig, G. (2024). WebArena: A realistic web environment for building autonomous agents. ICLR. Link: Open DOI / publication page

[17] Deng, X., Gu, Y., Zheng, B., Chen, S., Stevens, S., Wang, B., Sun, H., & Su, Y. (2023). Mind2Web: Towards a generalist agent for the web. Advances in Neural Information Processing Systems, 36. DOI: Open DOI / publication page

[18] Mialon, G., Fourrier, C., Swift, C., Wolf, T., LeCun, Y., & Scialom, T. (2023). GAIA: A benchmark for general AI assistants. ICLR. DOI: Open DOI / publication page

[19] Jimenez, C.E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., & Narasimhan, K.R. (2024). SWE-bench: Can language models resolve real-world GitHub issues? ICLR. Link: Open DOI / publication page

[20] Nakano, R., Hilton, J., Balaji, S., Wu, J., Ouyang, L., Kim, C., Hesse, C., Jain, S., Kosaraju, V., Cobbe, K., Eloundou, T., Krueger, G., Button, K., Knight, M., Chess, B., & Schulman, J. (2021). WebGPT: Browser-assisted question-answering with human feedback. arXiv. DOI: Open DOI / publication page

[21] Gao, T., Yen, H., Yu, J., & Chen, D. (2023). Enabling large language models to generate text with citations. Proceedings of EMNLP 2023, pp. 6465-6488. DOI: Open DOI / publication page

[22] Min, S., Krishna, K., Lyu, X., Lewis, M., Yih, W-t., Koh, P.W., Iyyer, M., Zettlemoyer, L., & Hajishirzi, H. (2023). FActScore: Fine-grained atomic evaluation of factual precision in long form text generation. Proceedings of EMNLP 2023, pp. 12076-12100. DOI: Open DOI / publication page

Note: All references are either to DOI resolvers or official conference/publisher pages. They may not always be accessible dependent on the publisher.